

ST Open Science Workshop
September 2024

Xpectra: A Python Spectral Analysis Package for Modeling Laboratory Spectra

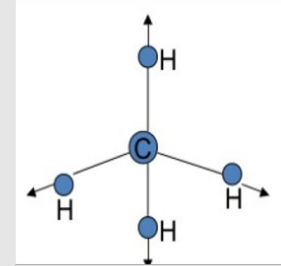
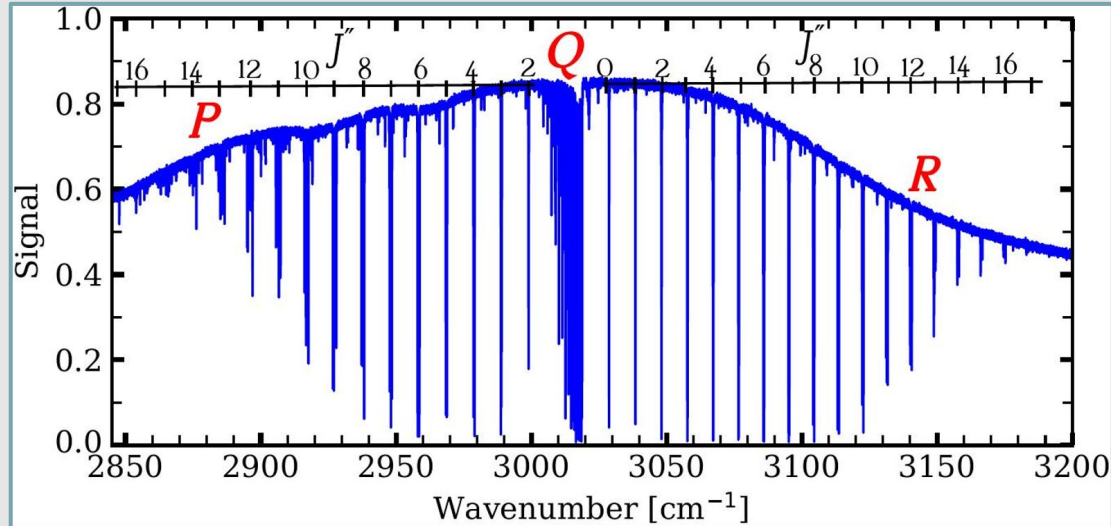
Emily Mader (STT), Ehsan (Sam) Gharib-Nezhad (STT), Salma Bejaoui (STA),
Natasha Batalha (STA), Richard Freedman (STT), Farid Salama (STA), Iouli Gordon (Harvard)

Key Objectives

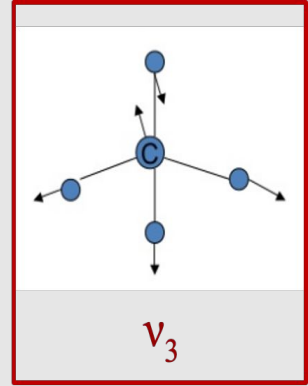
- Explore the Laboratory Spectra
 - ◆ Load and visualize the data
 - ◆ Enable interactive Search of molecular bands
- Spectral analysis
 - ◆ Fit the Baseline
 - ◆ Fit the spectral line profiles
 - ◆ Extract spectroscopic parameters
 - Line position
 - Pressure-broadening coefficients
 - Intensity
- Perform statistical tests on the fitting accuracy
- Identify and label quantum assignments starting from HITRAN database
- Report the results with their uncertainty for publication in Tables & figures

Case study: CH₄

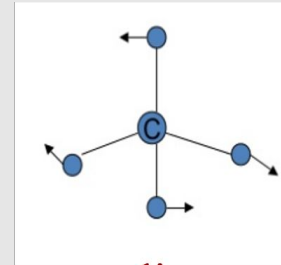
- Important absorber in exoplanetary and sub-stellar atmospheres
- Vibrational modes
 - ◆ Stretching: ν_1 (symmetric), ν_3 (asymmetric)
 - ◆ Bending: ν_2 , ν_4



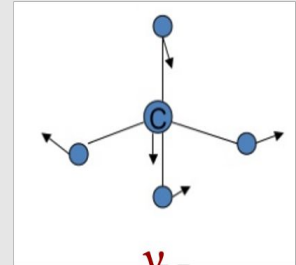
ν_1



ν_3



ν_2



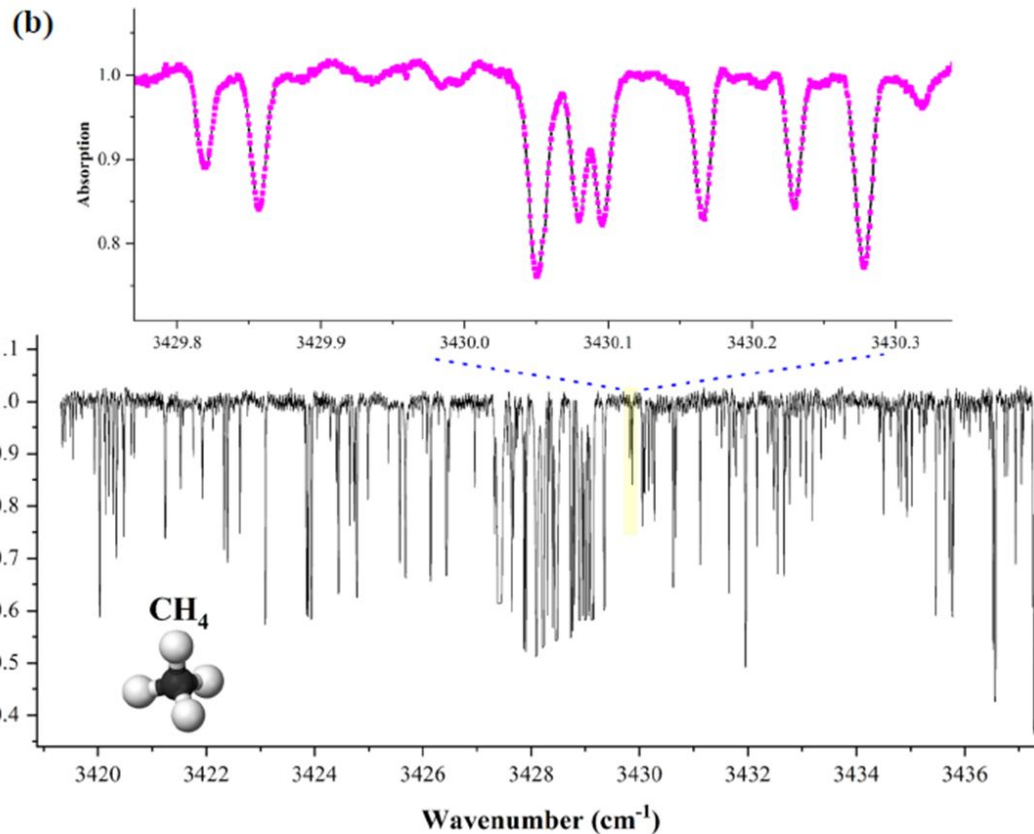
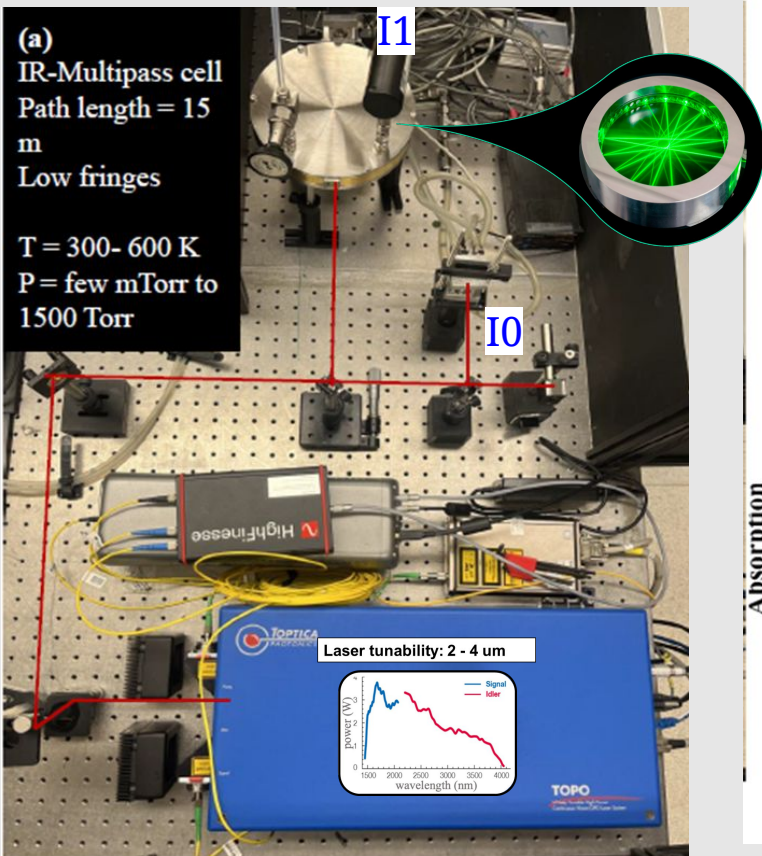
ν_4

Pressure Broadening Measurements of Key Exoplanetary Molecules

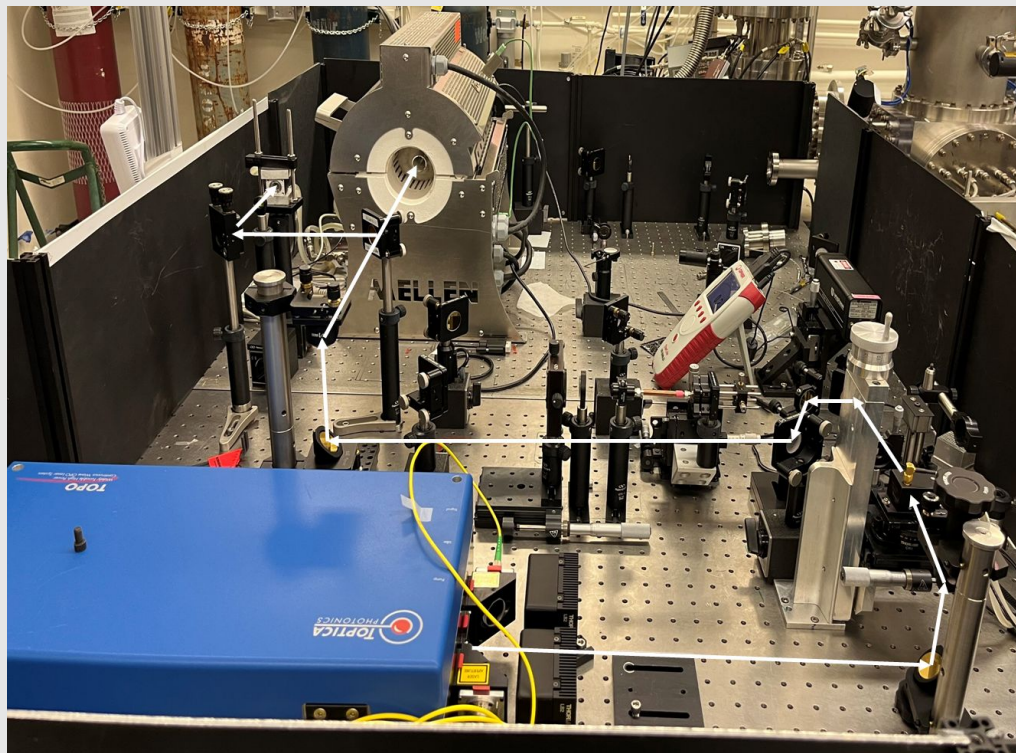
Multipass gas cell (300-600 K)

(a)
IR-Multipass cell
Path length = 15 m
Low fringes

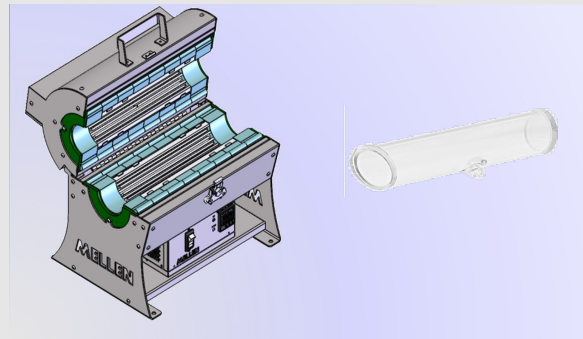
T = 300- 600 K
P = few mTorr to
1500 Torr



Pressure Broadening Measurements of Key Exoplanetary Molecules



Temperature range: 300-1250 K



				<i>Broadeners</i>		
		λ (μm)	Intensity	H ₂	He	CO ₂
<i>Absorbers</i>	CO ₂	2.68	4 10 ⁻²⁰	✓	✓	✓
		2.77	6 10 ⁻²⁰	✓	✓	✓
	CO	2.35	4 10 ⁻²¹	✓	✓	✓
	CH ₄	3.3	2 10 ⁻¹⁹		✓	✓
	HCN	3.03	3 10 ⁻¹⁹	✓	✓	

Xpectra Functionality



Module: SpecMaster

Prepare Spectra

Define range

Module: LineAssigner

Identify Lines

Quantum Assignment

Compare Literature

HITRAN API

Module: SpecFitAnalyzer

Fit Baseline

Model Peaks

Fit Line Profile

Calc. Broadening

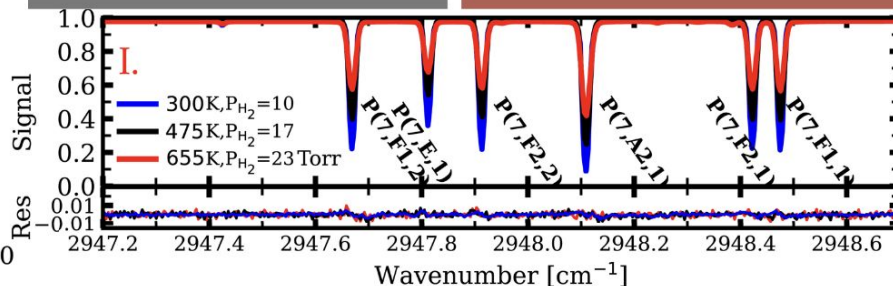
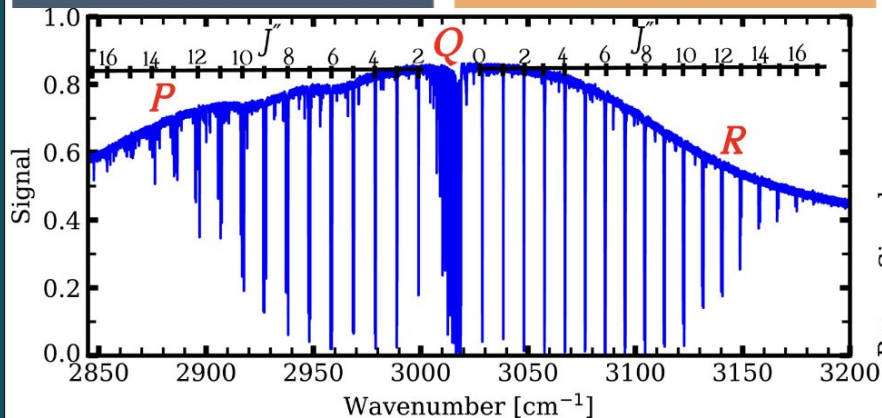
Calc. Intensity

Module: SpecStatVisualizer

Plot, Report, Print

Cal. Fitting Metrics

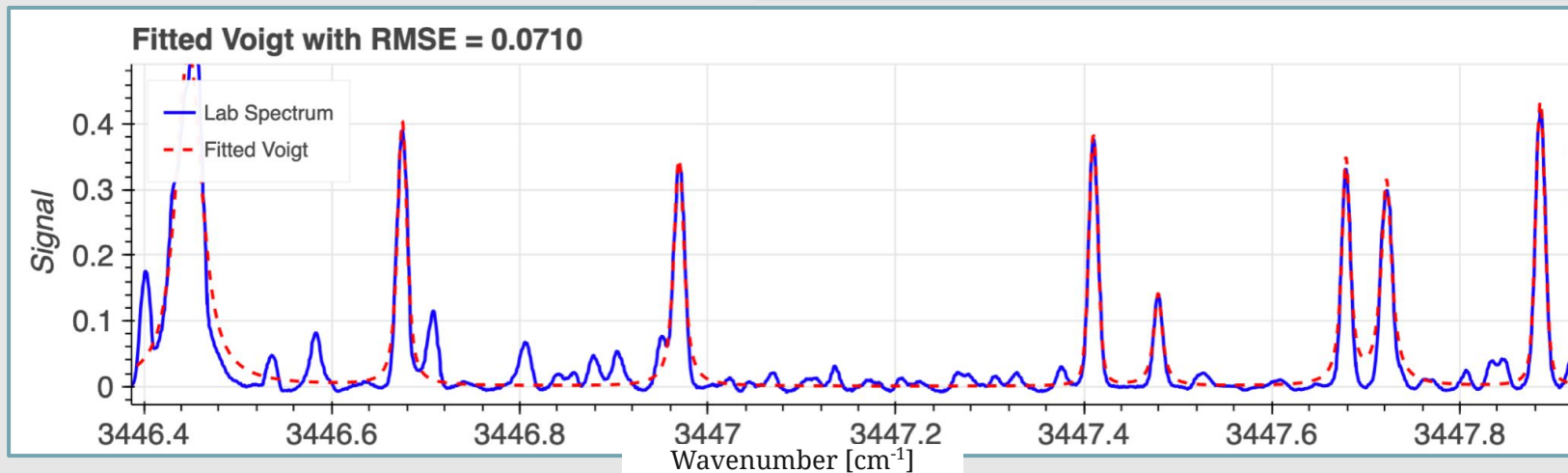
Interactive Vis.



Example of Line Profile Analysis

- Fit the observed lines in the data using a chosen line profile
- Print fitted parameters with error bars
- Visualize fit, calculate metrics

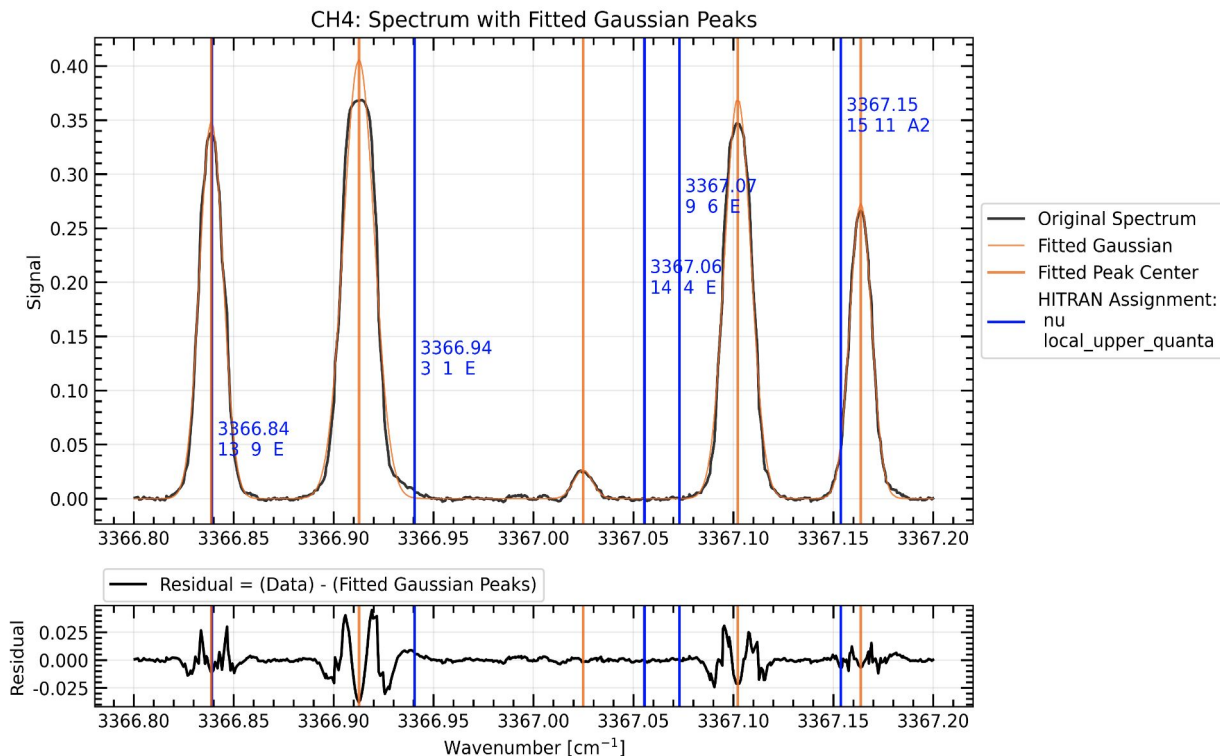
Peak Number	center	center Error	Intensity	Intensity Error	Width	Width Error
5	3446.4474	0.0007	0.491	0.0206	0.015	0.0007
6	3446.6749	0.0006	0.393	0.0340	0.006	0.0005
7	3446.9693	0.0007	0.329	0.0350	0.006	0.0007
8	3447.4102	0.0006	0.376	0.0361	0.005	0.0006
9	3447.4791	0.0015	0.138	0.0376	0.005	0.0015
10	3447.6789	0.0006	0.331	0.0369	0.005	0.0006
11	3447.7220	0.0008	0.305	0.0356	0.006	0.0008



Example of Line Assigner: Preliminary Demo

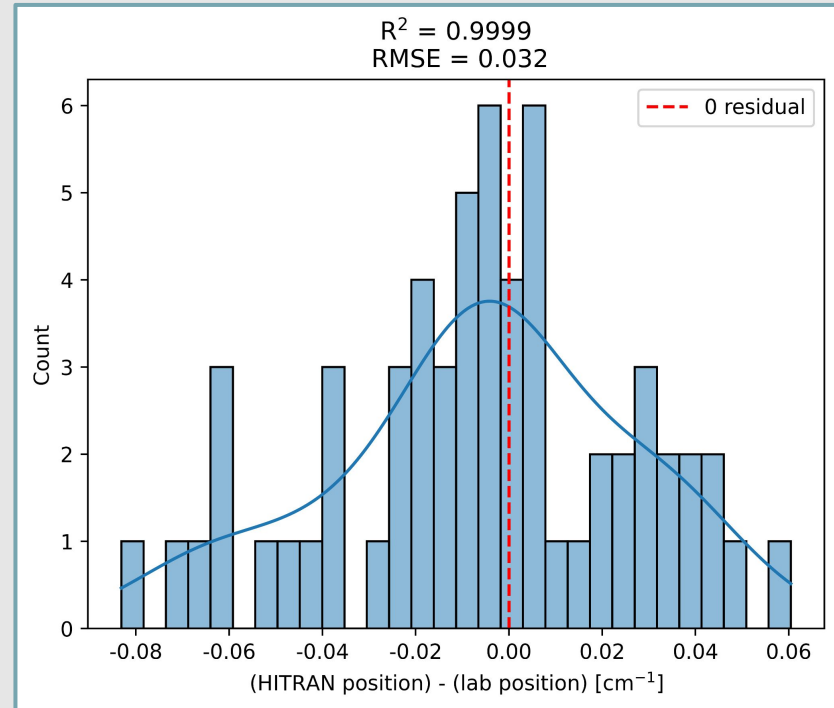
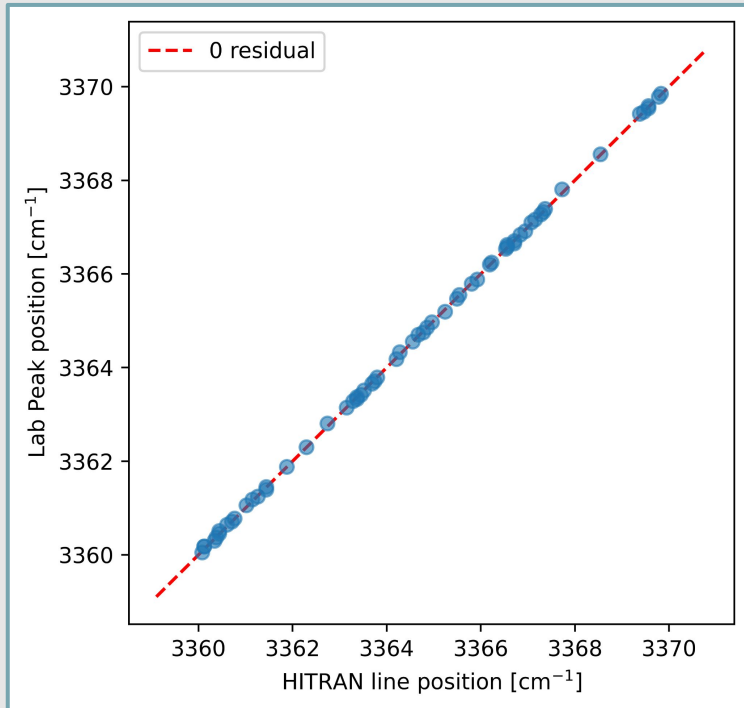
- *LineAssigner* module
- Select any spectroscopic parameters (HITRAN line list) to display
- Choose wavelength range
- Interactive visualization
- Enable further Spectral fitting

```
assign.hitran_line_assigner(wavenumber_range = wavenumber_range,  
                           columns_to_print = ["nu", "local_upper_quanta"],  
                           _plot_seaborn_ = True,)
```



Identified Line Analysis: Preliminary

- Assigning 50+ lines creates a sample for comparing lab lines to expected
- Calculate metrics, visualize distribution, analyze results



Baseline fitting:

Asymmetrically reweighted penalized least squares smoothing (ARPLS) Method

Baseline correction methods based on penalized least squares are successfully applied to various spectral analyses. The methods change the weights iteratively by estimating a baseline. If a signal is below a previously fitted baseline, large weight is given. On the other hand, no weight or small weight is given when a signal is above a fitted baseline as it could be assumed to be a part of the peak.

```
# Initialize signal and parameters
y = signal_values # Raw signal data
N = length(y) # Number of data points in the signal
D = difference_matrix(N) # Create difference matrix for smoothing
H = smoothing_penalty * transpose(D) * D # Calculate smoothing term

# Initialize weights
w = array_of_ones(N) # Start with equal weights

# Iterative fitting loop
for i in range(max_iterations):
    W = diagonal_matrix(w) # Create weight matrix
    WH = W + H # Combine weight matrix and smoothing penalty

    # Solve for the estimated baseline using LU decomposition
    z = solve_system(WH, w * y)

    # Calculate difference between the original signal and baseline estimate
    d = y - z

    # Find negative differences (outliers below the baseline)
    dn = find_negatives(d)

    # Update mean and standard deviation of these outliers
    m = mean(dn)
    s = standard_deviation(dn)

    # Update weights based on outlier information
    wt = 1 / (1 + exp(2 * (d - (2 * s - m)) / s))

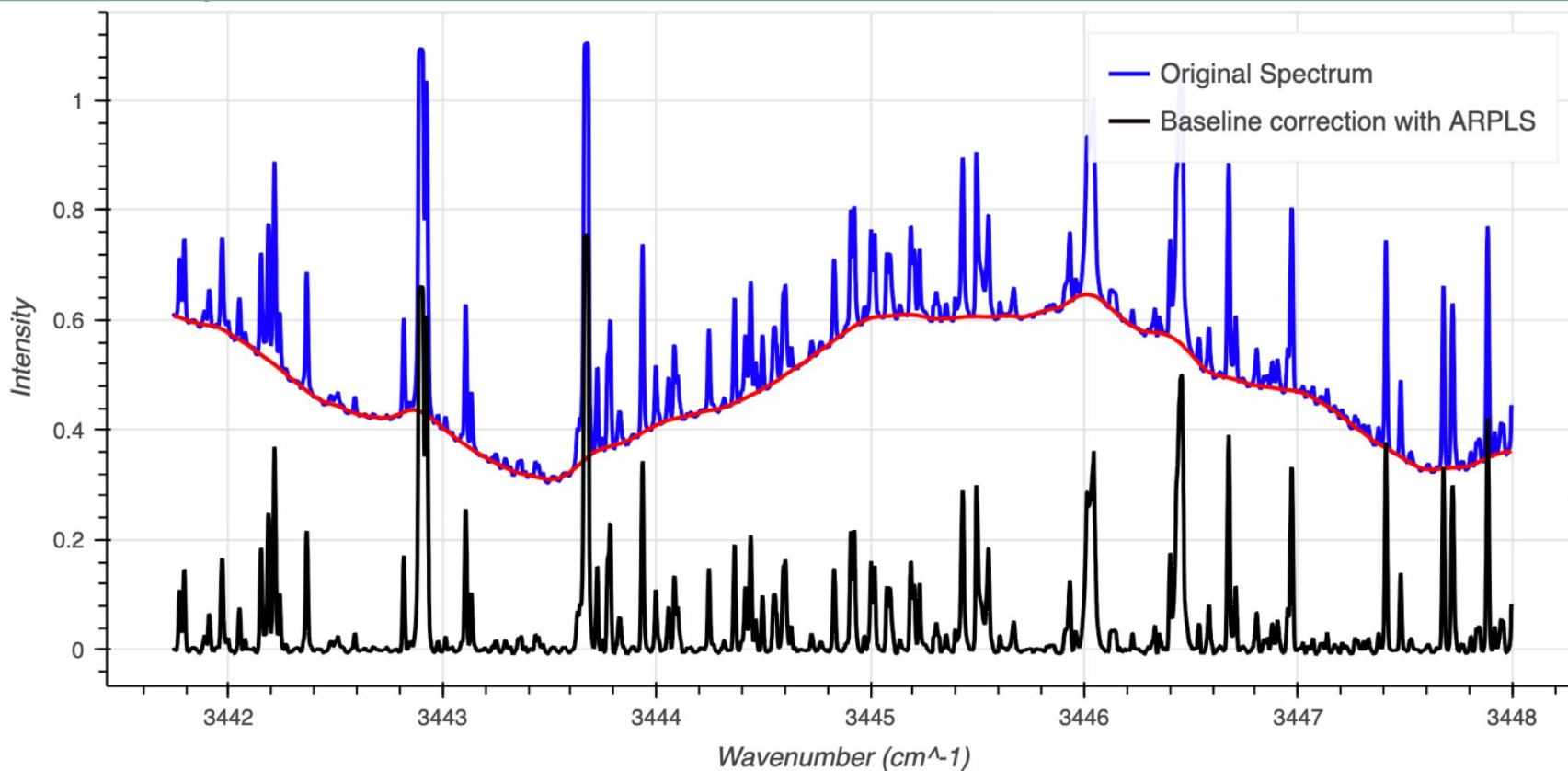
    # Check if the change in weights is small enough to stop
    if relative_change(w, wt) < threshold:
        break

    # Update weights for the next iteration
    w = wt

# Final corrected signal = original signal - estimated baseline
corrected_signal = y - z
```

Comments suggested by ChatGPT

Example of Baseline Correction - ARPLS



Why Open Science?

- *Reproducible, inclusive, accessible*
- Modules, datasets, and Jupyter notebooks are ‘open-source’ = made publicly available
- Encourages collaboration, empowers the community
- Science is more robust the more times it is repeated



Photo by fauxels from Pexels:
<https://www.pexels.com/photo/top-view-photo-of-people-handshaking-3182784/>

Programing Best Practices

Github

- Upload code to a shared space, collaboration
- Control versions and track history

Sphinx/Tutorial

- Compile a tutorial using Sphinx, based on docstrings and modules
- Guide users to deploy the package & develop it

Docstrings

- A detailed/concise description of a function or class
- Specifies input and output parameters

I/O Data Types

- Pre-defined input and output format
- “Type hints”
- Informs users of the correct data types to use

Naming Conventions

- Following convention is essential for readability
- Class vs. function
- PascalCase or underscore_notation

Test Cases

- It is imperative to test a package with real data
- Testing exposes errors, more tests, the more robust

Python Standards: Docstring

→ NumPy style docstrings

Function name in lowercase,
words separated by underscores

```
fit_spectrum(initial_guesses: Union[list, ndarray], line_profile: str = 'gaussian', fitting_method: str = 'lm',  
wavenumber_range: Optional[Union[list, tuple, ndarray]] = None, __plot_bokeh__: bool = True,  
__plot_seaborn__: bool = False, __print__: bool = True) → None
```

Short
description

Fit a spectrum with multiple peaks using specified line profiles (gaussian, lorentzian, voigt).

Input

Parameters:

- **initial_guesses** (*list or np.ndarray*) – List of initial guesses for parameters of the line profile.
- **line_profile** (*str, {'gaussian', 'lorentzian', 'voigt'}, optional*) – Type of line profile to use for fitting. Default is 'gaussian'.
- **wavenumber_range** (*list-like, optional*) – List-like object (list, tuple, or np.ndarray) with of length 2 representing wavenumber range to fit within.
- **__plot_bokeh__** (*bool*) – True or False.
- **__plot_seaborn__** (*bool*) – True or False.
- **__print__** (*bool*) – True or False.

Type
specified

Output

Returns:

fitted_params – List of fitted parameters for each peak.

Return type:

list

Python Standards: Logging

- Provide error messages using the logging standard library module
- 5 log levels: DEBUG, INFO, WARNING, ERROR, and CRITICAL
- Messages must be simple, descriptive, and *helpful*

The script:

Configure messages

```
import logging
logging.basicConfig(level=logging.WARNING, format='%(levelname)s: %(message)s')

# Define y
try:
    y = self.y_baseline_corrected
except AttributeError:
    logging.warning("'y_baseline_corrected' attribute does not exist.
    Using baseline-included 'signal_values' attribute instead.")
    y = self.signal_values

# Define line_profile_func
if line_profile == 'gaussian':
    line_profile_func = self.gaussian
elif line_profile == 'lorentzian':
    line_profile_func = self.lorentzian
elif line_profile == 'voigt':
    line_profile_func = self.voigt
else:
    logging.critical(f"Unknown line profile: {line_profile}." + "
    Please choose one: {'gaussian', 'lorentzian', 'voigt'}.")
    return
```

Error messages set to trigger under certain conditions

The notebook:

Did not run signal correction first

Input an invalid argument

```
instance = SpecFitAnalyzer(
    wavenumber_values = np.array([1,2,3]),
    signal_values = np.array([1,2,3])
)
```

```
instance.fit_spectrum(initial_guesses = [],
    line_profile = 'Bananas')
```

```
WARNING: 'y_baseline_corrected' attribute does not exist.
Using baseline-included 'signal_values' attribute instead.
CRITICAL: Unknown line profile: Bananas. Please choose on
e: {'gaussian', 'lorentzian', 'voigt'}.
```

Conditions met, messages triggered

Impact of Xpectra on code ST Research:

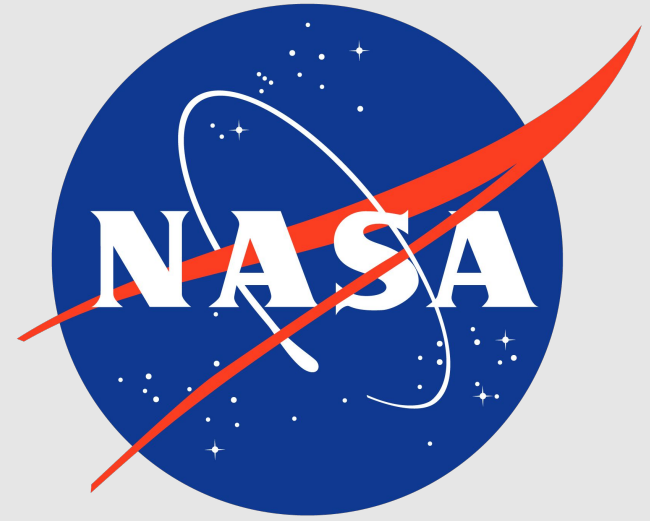
- Salma Bejoui - Fitting Laser spectra
- Michel Neveu - Spectral analysis by adding a module for UV/Vis regime
- Sarah Nickerson - Identify/Assign Atomic and Molecular lines from Galaxy spectra



NASA/SOFIA/Dylan O'Donnell

Conclusion/Outlook

- In development of Xpectra, we utilize:
 - ◆ Programming best practices,
 - ◆ Spectral optimization algorithms,
 - ◆ & Open science principles
- Continued collaboration with code ST



Developed as a part of the XRP grant: “From Super-Earths to Sub-Neptunes: Laboratory Measurements of High-Temperature Absorption Cross Sections and Pressure-broadening coefficients of key Exoplanetary molecules with High Resolution Mid-IR Spectroscopy”

EGN and EM would like to thank OSTEM internships and funding through the NASA, award AL-67060